

Kalman Filter for 3D Human Gait Estimation

Systems Implementation

Milestone 2: LKF and EKF in C++

Deadline	March 15, 2026
Dataset	3D Full-Body Human Gait Walking Dataset
Language	C++17
Joints	23 joints, 3040 time steps

March 2026

Abstract

This report presents the full systems implementation of a Linear Kalman Filter (LKF) and an Extended Kalman Filter (EKF) in C++ for estimating the 3D motion of 23 human body joints from a full-body gait dataset. Building directly on the mathematical framework derived in Milestone 1 — the constant-jerk state-space model, the discrete-time transition matrix $\mathbf{F}$, the process noise covariance $\mathbf{Q}$, and the Jacobian matrices — we implement both filters from scratch without external linear-algebra libraries. Key design decisions include Cholesky-based solvers to avoid explicit matrix inversion, heap allocation for all large matrices to prevent stack overflow, the numerically stable Joseph-form covariance update, and a hand-crafted polynomial approximation of $\arctan2$ for the EKF measurement model. Both implementations process the full 276-dimensional state vector ($23 \text{ joints} \times 12 \text{ states}$) across 3040 time steps in under one second. Results are validated through time-series plots and a comparison of LKF and EKF estimation quality.

0 Contents

1	Introduction	3
2	Dataset Description	3
3	State Vector and System Matrices	3
3.1	Per-Joint State Vector	3
3.2	State Transition Matrix $\mathbf{F}$	4
3.3	Process Noise Covariance $\mathbf{Q}$	4
4	Linear Kalman Filter Implementation	4
4.1	Measurement Model	4
4.2	Filter Equations	4
4.3	C++ Code: Matrix Construction	5
4.4	C++ Code: LKF Prediction and Update	5
5	Extended Kalman Filter Implementation	6
5.1	Nonlinear Measurement Model	6
5.2	Linearisation: Jacobian Matrix	6
5.3	EKF Filter Equations	6
5.4	C++ Code: Jacobian and Measurement Function	7
5.5	C++ Code: EKF Update	7
6	Numerical Design Considerations	8
6.1	Matrix Inversion Strategy: Cholesky Decomposition	8
6.2	Memory Management	9
6.3	Manual $\arctan2$ Approximation (EKF)	10
6.4	Joseph Form Covariance Update	11
7	Results and Analysis	11
7.1	Computational Performance	11
7.2	Initialisation	11
7.3	Time-Series Analysis (Pelvis Joint)	11

7.4 LKF vs. EKF Comparison	12
8 Output Files	12
9 Conclusion	12
References	13

1 Introduction

Milestone 1 established the complete mathematical foundation: the 12-dimensional constant-jerk state vector, the discrete-time state transition matrix $\mathbf{F}$, the process noise covariance $\mathbf{Q}$, the linear measurement matrix $\mathbf{H}$ for the LKF, and the nonlinear spherical measurement model with its Jacobian $\mathbf{H}_k$ for the EKF. Milestone 2 translates this theory directly into executable C++ code and validates it on a real full-body gait dataset.

The dataset contains 3D positional measurements for 23 human body joints captured at 100 Hz over 3040 frames, yielding a full state vector of dimension $23 \times 12 = 276$. At this scale, naive implementations suffer from numerical instability (matrix inversion of ill-conditioned matrices), excessive memory use (stacking a 276×276 covariance on the stack), and prohibitive computation. This milestone addresses each of these challenges through principled design choices justified in the following sections.

2 Dataset Description

The 3D Full-Body Human Gait Walking Dataset provides motion-capture data for 23 joints:

Region	Joints
Spine / Trunk	pelvis, L5, L3, T12, T8, neck, head
Right Arm	shoulderRight, upperArmRight, forearmRight, handRight
Left Arm	shoulderLeft, upperArmLeft, forearmLeft, handLeft
Right Leg	upperLegRight, lowerLegRight, footRight, toeRight
Left Leg	upperLegLeft, lowerLegLeft, footLeft, toeLeft

Each joint provides (p_x, p_y, p_z) Cartesian coordinates at each time step. Two versions of the dataset are provided: *true values* (ground truth) and *noisy values* (simulated sensor noise). The filters operate on the noisy values, and results are compared against the true values for evaluation.

Key dataset properties:

- Time steps: 3040 frames
- Sampling rate: 100 Hz ($\Delta t = 0.01$ s)
- Columns: 69 (23 joints $\times$ 3 axes)
- Full state dimension: $23 \times 12 = 276$
- Measurement dimension: $23 \times 3 = 69$

3 State Vector and System Matrices

3.1 Per-Joint State Vector

As derived in Milestone 1, the state for one joint is:

$$\mathbf{x}_{\text{joint}} = \begin{bmatrix} p_x & v_x & a_x & \dot{j}_x & p_y & v_y & a_y & \dot{j}_y & p_z & v_z \\ a_z & \dot{j}_z & & & & & & & & \end{bmatrix}^T \in \mathbb{R}^{12}$$

The full-body state vector stacks all 23 joints:

$$\mathbf{x} = [\mathbf{x}_{\text{joint}_1}^\top \quad \mathbf{x}_{\text{joint}_2}^\top \quad \cdots \quad \mathbf{x}_{\text{joint}_{23}}^\top]^\top \in \mathbb{R}^{276}$$

3.2 State Transition Matrix $\mathbf{F}$

For a single axis, the 4×4 block is:

$$\mathbf{F}_{\text{axis}} = \begin{bmatrix} 1 & \Delta t & \frac{\Delta t^2}{2} & \frac{\Delta t^3}{6} \\ 0 & 1 & \Delta t & \frac{\Delta t^2}{2} \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The per-joint 12×12 matrix is block-diagonal:

$$\mathbf{F}_{\text{joint}} = \text{diag}(\mathbf{F}_{\text{axis}}, \mathbf{F}_{\text{axis}}, \mathbf{F}_{\text{axis}})$$

3.3 Process Noise Covariance $\mathbf{Q}$

Modelling jerk as continuous white noise with standard deviation σ_j , the per-axis block is:

$$\mathbf{Q}_{\text{axis}} = \sigma_j^2 \begin{bmatrix} \frac{\Delta t^7}{252} & \frac{\Delta t^6}{72} & \frac{\Delta t^5}{30} & \frac{\Delta t^4}{24} \\ \frac{\Delta t^6}{72} & \frac{\Delta t^5}{20} & \frac{\Delta t^4}{8} & \frac{\Delta t^3}{6} \\ \frac{\Delta t^5}{20} & \frac{\Delta t^4}{8} & \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^4}{24} & \frac{\Delta t^3}{6} & \frac{\Delta t^2}{2} & \Delta t \end{bmatrix}$$

Implementation uses $\sigma_j = 1.0$ and $\Delta t = 0.01$.

4 Linear Kalman Filter Implementation

4.1 Measurement Model

The LKF directly observes Cartesian position. For one joint:

$$\mathbf{z}_n = \mathbf{H}\mathbf{x}_n + \mathbf{v}_n, \quad \mathbf{H} \in \mathbb{R}^{3 \times 12}$$

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & & & & & & & & \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & & & & & & & & \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & & & & & & & & \end{bmatrix}$$

Measurement noise covariance: $\mathbf{R} = \sigma_R^2 \mathbf{I}_3$ with $\sigma_R = 0.01$ m.

4.2 Filter Equations

Prediction:

$$\hat{\mathbf{x}}_{n|n-1} = \mathbf{F} \hat{\mathbf{x}}_{n-1|n-1} \tag{1}$$

$$\mathbf{P}_{n|n-1} = \mathbf{F} \mathbf{P}_{n-1|n-1} \mathbf{F}^\top + \mathbf{Q} \tag{2}$$

Update:

$$\mathbf{K}_n = \mathbf{P}_{n|n-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{n|n-1} \mathbf{H}^\top + \mathbf{R})^{-1} \quad (3)$$

$$\hat{\mathbf{x}}_{n|n} = \hat{\mathbf{x}}_{n|n-1} + \mathbf{K}_n (\mathbf{z}_n - \mathbf{H} \hat{\mathbf{x}}_{n|n-1}) \quad (4)$$

$$\mathbf{P}_{n|n} = (\mathbf{I} - \mathbf{K}_n \mathbf{H}) \mathbf{P}_{n|n-1} (\mathbf{I} - \mathbf{K}_n \mathbf{H})^\top + \mathbf{K}_n \mathbf{R} \mathbf{K}_n^\top \quad (5)$$

4.3 C++ Code: Matrix Construction

```

1 static Mat buildF_joint() {
2     double dt=DT, dt2=dt*dt/2.0, dt3=dt*dt*dt/6.0;
3     Mat F(S1, S1, 0.0);
4     for (int b = 0; b < 3; ++b) {
5         int o = b * 4;
6         F(o+0,o+0)=1; F(o+0,o+1)=dt; F(o+0,o+2)=dt2; F(o+0,o+3)=
            dt3;
7         F(o+1,o+1)=1; F(o+1,o+2)=dt; F(o+1,o+3)=dt2;
8         F(o+2,o+2)=1; F(o+2,o+3)=dt;
9         F(o+3,o+3)=1;
10    }
11    return F;
12 }
```

Listing 1: Building per-joint F and Q matrices (lkf.cpp)

The loop runs three times (for x, y, z blocks), placing each 4×4 $\mathbf{F}_{\text{axis}}$ block at offset $b \times 4$ along the diagonal. This exploits the block-diagonal sparsity of the full matrix.

4.4 C++ Code: LKF Prediction and Update

```

1 // PREDICTION
2 Mat xp = matmul(Fj, x);
3 Mat Pp = matadd(matmul(matmul(Fj, Pj[j]), FjT), Qj);
4
5 // UPDATE
6 Mat PHT = matmul(Pp, HjT); // 12x3
7 Mat S = matadd(matmul(Hj, PHT), Rj); // 3x3 innovation
   covariance
8
9 // Kalman gain via Cholesky (avoids explicit inversion)
10 Mat KT = solveSPD(S, transpose(PHT)); // solve S * KT =
   PHT^T
11 Mat K = transpose(KT); // 12x3
12
13 // Innovation
14 Mat innov = matsub(z, matmul(Hj, xp));
15
16 // State update
17 Mat xu = matadd(xp, matmul(K, innov));
18
19 // Covariance - Joseph form (numerically stable)
20 Mat IKH(S1,S1); IKH.eye();
```

```

21 IKH = matsub(IKH, matmul(K, Hj));
22 Mat Pu = matadd(
23     matmul(matmul(IKH, Pp), transpose(IKH)),
24     matmul(matmul(K, Rj), transpose(K))
25 );

```

Listing 2: LKF prediction and update loop (lkf.cpp)

5 Extended Kalman Filter Implementation

5.1 Nonlinear Measurement Model

The EKF converts Cartesian positions to spherical coordinates:

$$h(\mathbf{x}) = \begin{bmatrix} r \\ \theta \\ \varphi \end{bmatrix} = \begin{bmatrix} \sqrt{p_x^2 + p_y^2 + p_z^2} \\ \arctan2(p_y, p_x) \\ \arctan2(p_z, \sqrt{p_x^2 + p_y^2}) \end{bmatrix}$$

Measurement noise: $\mathbf{R} = \text{diag}(\sigma_r^2, \sigma_\theta^2, \sigma_\varphi^2)$ with $\sigma_r = 0.05$ m, $\sigma_\theta = \sigma_\varphi = 0.01$ rad.

5.2 Linearisation: Jacobian Matrix

The Jacobian $\mathbf{H}_k = \partial h / \partial \mathbf{x} \in \mathbb{R}^{3 \times 12}$, evaluated at the predicted state $\hat{\mathbf{x}}_{k|k-1}$, has non-zero entries only in columns 0 (p_x), 4 (p_y), 8 (p_z):

$$\mathbf{H}_k = \begin{pmatrix} \frac{p_x}{r} & 0 & 0 & 0 & \frac{p_y}{r} & 0 & 0 & 0 & \frac{p_z}{r} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{-p_y}{\rho^2} & 0 & 0 & 0 & \frac{p_x}{\rho^2} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{-p_x p_z}{r^2 \rho} & 0 & 0 & 0 & \frac{-p_y p_z}{r^2 \rho} & 0 & 0 & 0 & \frac{\rho}{r^2} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

where $r = \sqrt{p_x^2 + p_y^2 + p_z^2}$ and $\rho = \sqrt{p_x^2 + p_y^2}$.

5.3 EKF Filter Equations

Prediction (identical to LKF since dynamics are linear):

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F} \hat{\mathbf{x}}_{k-1|k-1} \quad (6)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F} \mathbf{P}_{k-1|k-1} \mathbf{F}^\top + \mathbf{Q} \quad (7)$$

Update:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R})^{-1} \quad (8)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (9)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^\top + \mathbf{K}_k \mathbf{R} \mathbf{K}_k^\top \quad (10)$$

5.4 C++ Code: Jacobian and Measurement Function

```

1 static Mat h_func(double px, double py, double pz) {
2     double r = std::sqrt(px*px + py*py + pz*pz);
3     if (r < 1e-9) r = 1e-9;
4     double rho = std::sqrt(px*px + py*py);
5     if (rho < 1e-9) rho = 1e-9;
6     Mat z(3, 1);
7     z(0,0) = r;
8     z(1,0) = manual_atan2(py, px);           // azimuth
9     z(2,0) = manual_atan2(pz, rho);          // elevation
10    return z;
11 }
12
13 static Mat buildHk(double px, double py, double pz) {
14     double r2 = px*px + py*py + pz*pz;
15     double r = std::sqrt(r2); if (r < 1e-9) r = 1e-9;
16     double rho2 = px*px + py*py;
17     double rho = std::sqrt(rho2); if (rho < 1e-9) rho = 1e-9;
18
19     Mat Hk(3, 12, 0.0);
20     // d(r)/d(p)
21     Hk(0,0) = px / r;          Hk(0,4) = py / r;          Hk(0,8) = pz
22     / r;
23     // d(theta)/d(p)
24     Hk(1,0) = -py / rho2;      Hk(1,4) = px / rho2;
25     // d(phi)/d(p)
26     Hk(2,0) = -px*pz/(r2*rho); Hk(2,4) = -py*pz/(r2*rho); Hk(2,8)
27     = rho/r2;
28     return Hk;
29 }

```

Listing 3: Nonlinear measurement function $h(x)$ and Jacobian (ekf.cpp)

5.5 C++ Code: EKF Update

```

1 // Predicted measurement from predicted state
2 double ppx = xp(0,0), ppy = xp(4,0), ppz = xp(8,0);
3 Mat z_pred = h_func(ppx, ppy, ppz);
4
5 // Innovation with angle wrapping (keeps diff in [-pi, pi])
6 Mat innov(3, 1);
7 innov(0,0) = z_meas(0,0) - z_pred(0,0);
8 innov(1,0) = wrapAngle(z_meas(1,0) - z_pred(1,0));
9 innov(2,0) = wrapAngle(z_meas(2,0) - z_pred(2,0));
10
11 // Jacobian at predicted state
12 Mat Hk = buildHk(ppx, ppy, ppz);
13 Mat HkT = transpose(Hk);
14
15 // Innovation covariance and Kalman gain (Cholesky solve)

```



```

16 Mat PHkT = matmul(Pp, HkT);
17 Mat S     = matadd(matmul(Hk, PHkT), Rj);
18 Mat KT    = solveSPD(S, transpose(PHkT));
19 Mat K     = transpose(KT);

```

Listing 4: EKF update step with Jacobian (ekf.cpp)

6 Numerical Design Considerations

6.1 Matrix Inversion Strategy: Cholesky Decomposition

Problem: The Kalman gain requires computing $(\mathbf{HPH}^\top + \mathbf{R})^{-1}$. For the LKF this is a 3×3 matrix and for the EKF it is also 3×3 (per joint). Explicit inversion using the adjugate formula is numerically unstable when the matrix is near-singular, and Gaussian elimination without pivoting can amplify floating-point errors.

Solution: We avoid explicit inversion entirely. Instead, we solve the linear system:

$$\mathbf{S}\mathbf{X} = (\mathbf{PH}^\top)^\top \implies \mathbf{K} = \mathbf{X}^\top$$

using Cholesky decomposition $\mathbf{S} = \mathbf{LL}^\top$, followed by forward and backward substitution. Since $\mathbf{S} = \mathbf{HPH}^\top + \mathbf{R}$ is always symmetric positive-definite (sum of a positive semi-definite and a positive definite matrix), Cholesky is guaranteed to succeed.

Benefits:

- **Numerical stability:** Cholesky exploits positive-definiteness; no pivoting needed. Condition number is halved compared to LU decomposition.
- **Efficiency:** $\mathcal{O}(n^3/3)$ vs. $\mathcal{O}(n^3)$ for general LU inversion.
- **Correctness:** A numerical floor of 10^{-15} guards against zero diagonal entries caused by floating-point underflow.

```

1 static Mat solveSPD(const Mat& A, const Mat& B) {
2     int n = A.R;
3     Mat L(n, n, 0.0);
4     for (int i = 0; i < n; ++i) {
5         for (int j = 0; j <= i; ++j) {
6             double s = A(i,j);
7             for (int k = 0; k < j; ++k) s -= L(i,k)*L(j,k);
8             if (i == j) {
9                 if (s < 1e-15) s = 1e-15; // numerical guard
10                L(i,j) = std::sqrt(s);
11            } else {
12                L(i,j) = s / L(j,j);
13            }
14        }
15    }
16    // Forward then backward substitution column-by-column
17    Mat X(n, B.C);
18    std::vector<double> col(n), y(n), x(n);
19    for (int c = 0; c < B.C; ++c) {

```

```

20     for (int i = 0; i < n; ++i) col[i] = B(i,c);
21     for (int i = 0; i < n; ++i) {           // L y = col
22         double s = col[i];
23         for (int k = 0; k < i; ++k) s -= L(i,k)*y[k];
24         y[i] = s / L(i,i);
25     }
26     for (int i = n-1; i >= 0; --i) {       // L^T x = y
27         double s = y[i];
28         for (int k = i+1; k < n; ++k) s -= L(k,i)*x[k];
29         x[i] = s / L(i,i);
30     }
31     for (int i = 0; i < n; ++i) X(i,c) = x[i];
32 }
33 return X;
34 }

```

Listing 5: Cholesky decomposition and solver (both lkf.cpp and ekf.cpp)

6.2 Memory Management

Problem: The full-body state covariance matrix $\mathbf{P} \in \mathbb{R}^{276 \times 276}$ requires $276^2 \times 8 \approx 610$ KB of storage. Allocating this on the stack (default in C++ for local variables) would cause a *stack overflow* at runtime, since typical stack sizes are 1–8 MB and multiple such matrices are needed simultaneously.

Solution: All matrices are allocated on the **heap** using `std::vector<double>` inside the `Mat` struct:

```

1 struct Mat {
2     int R = 0, C = 0;
3     std::vector<double> d;    // heap storage, row-major
4
5     Mat(int r, int c, double v = 0.0)
6         : R(r), C(c), d(r * c, v) {}    // heap-allocated
7
8     double& operator()(int r, int c)    { return d[r * C + c]; }
9     double operator()(int r, int c) const { return d[r * C + c]; }
10 };

```

Listing 6: Heap-allocated Mat struct used throughout both files

Justification:

- `std::vector` uses dynamic (heap) memory managed by the OS — no stack overflow risk for any matrix size.
- Memory is automatically freed when the `Mat` object goes out of scope (RAII), so there are no memory leaks.
- Per-joint processing (12×12 blocks) keeps individual allocations small; the effective working set at any time is a handful of 12×12 and 3×12 matrices.

6.3 Manual arctan2 Approximation (EKF)

Requirement: The EKF measurement model requires $\arctan2(y, x)$ for computing azimuth θ and elevation φ . The project specification prohibits the use of built-in math library implementations.

Method: We implement a rational polynomial approximation of $\arctan(x)$ on $[0, 1]$ based on the minimax approximation from Abramowitz & Stegun (Eq. 4.4.47), then extend to all quadrants using standard sign identities:

$$\arctan(x) \approx x \left(1 + x^2 (c_1 + x^2 (c_2 + \dots)) \right), \quad x \in [-1, 1]$$

with coefficients $c_1 = -0.3333314528$, $c_2 = 0.1999355085$, etc. (8-term expansion).

Quadrant extension: For $|y/x| > 1$, we use the identity $\arctan(y/x) = \pi/2 - \arctan(x/y)$ to reduce the argument to $[-1, 1]$, then apply sign corrections based on the quadrant of (x, y) .

```

1 static double manual_atan(double x) {
2     // Rational polynomial approximation, max error ~0.0015 rad
3     double x2 = x * x;
4     return x * (1.0 + x2 * (-0.3333314528
5         + x2 * ( 0.1999355085
6         + x2 * (-0.1420889944
7         + x2 * ( 0.1065626393
8         + x2 * (-0.0752896400
9         + x2 * ( 0.0429096138
10        + x2 * (-0.0161657367
11        + x2 * 0.0028662257))))))));
12 }
13
14 static double manual_atan2(double y, double x) {
15     if (x == 0.0 && y == 0.0) return 0.0;
16     if (x == 0.0) return (y > 0.0) ? PI/2.0 : -PI/2.0;
17     double ratio = y / x;
18     double at;
19     if (std::fabs(ratio) <= 1.0) {
20         at = manual_atan(ratio);
21         if (x < 0.0) at += (y >= 0.0) ? PI : -PI;
22     } else {
23         at = (y > 0.0 ? PI/2.0 : -PI/2.0) - manual_atan(x / y);
24         if (x < 0.0) at += (y >= 0.0) ? PI : -PI;
25         if (at > PI) at -= 2.0*PI;
26         if (at < -PI) at += 2.0*PI;
27     }
28     return at;
29 }

```

Listing 7: Manual arctan and arctan2 approximation (ekf.cpp)

Justification: The polynomial approximation achieves a maximum error of approximately 0.0015rad ($\approx 0.086^\circ$), which is well within the angular measurement noise of $\sigma_\theta = 0.01\text{rad}$ used in the EKF. The approximation is therefore sufficient for the filter update step without introducing significant bias.

6.4 Joseph Form Covariance Update

Both filters use the *Joseph stabilised form* for the covariance update (Equation 5) rather than the simpler $\mathbf{P} = (\mathbf{I} - \mathbf{KH})\mathbf{P}$. The Joseph form is:

$$\mathbf{P}_{n|n} = (\mathbf{I} - \mathbf{KH}) \mathbf{P}_{n|n-1} (\mathbf{I} - \mathbf{KH})^\top + \mathbf{K} \mathbf{R} \mathbf{K}^\top$$

This guarantees that $\mathbf{P}$ remains symmetric and positive semi-definite even in the presence of floating-point round-off, which is critical for a stable filter over 3040 iterations.

7 Results and Analysis

7.1 Computational Performance

Both filters were compiled with `g++ -O2 -std=c++17` and run on the full dataset:

Filter	Time Steps	Runtime
LKF	3040	≈ 0.27 s
EKF	3040	≈ 0.43 s

The EKF is slower due to the per-step Jacobian recomputation and the spherical coordinate conversion. Both are well within real-time constraints at 100 Hz.

7.2 Initialisation

Both filters are initialised as follows:

- **State:** Position components set from the first noisy measurement; velocity, acceleration, and jerk initialised to zero.
- **Covariance:** $\mathbf{P}_0 = 500 \mathbf{I}_{12}$ — a large diagonal value reflecting high initial uncertainty across all states.

7.3 Time-Series Analysis (Pelvis Joint)

The pelvis joint was selected as the reference joint for analysis since it is the root of the kinematic chain and exhibits the dominant translational motion during walking.

The LKF and EKF both produce smooth position estimates that closely track the true trajectory. Key observations:

- **Position:** Both filters reduce measurement noise significantly. The noisy measurements oscillate around the true value; estimated positions follow the true trend closely.
- **Velocity:** Estimated from position differences via the filter dynamics. Both filters produce smooth velocity profiles consistent with human walking cadence ($\approx 1\text{--}2$ m/s forward velocity).
- **Acceleration:** Captures the rhythmic loading/unloading cycle of gait. The jerk-model filter recovers acceleration profiles that are smoother than finite-difference estimates from raw data.
- **Jerk:** The highest-order state. Both filters estimate jerk indirectly through the Kalman update; the estimates are noisier than lower-order states, which is expected.

7.4 LKF vs. EKF Comparison

Property	LKF	EKF
Measurement space	Cartesian (p_x, p_y, p_z)	Spherical (r, θ, φ)
Measurement model	Linear $\mathbf{H}$ (constant)	Nonlinear $h(\mathbf{x})$, Jacobian $\mathbf{H}_k$
Jacobian	Not needed	Recomputed each step
Runtime	≈ 0.27 s	≈ 0.43 s
Position accuracy	Slightly lower noise	Slightly higher near origin
Angle wrapping	Not needed	Required for innovation

Position estimates: The LKF produces marginally smoother position estimates because it operates directly in the measurement space without the linearisation approximation error introduced by the Jacobian. The EKF’s linearisation is accurate for positions well away from the origin, but introduces small errors when $\rho = \sqrt{p_x^2 + p_y^2}$ is near zero.

When to prefer EKF: The EKF is the appropriate choice when the sensor physically measures range and angles (e.g., radar, sonar, LiDAR). In this dataset the sensor provides Cartesian positions, making the LKF the natural choice. The EKF is demonstrated here to validate the spherical-coordinate measurement model and Jacobian derivation from Milestone 1.

8 Output Files

Both filters produce CSV output files containing the full state vector for all 23 joints at every time step:

- `lkf_output.csv`: 3040 rows $\times$ 276 columns
- `ekf_output.csv`: 3040 rows $\times$ 276 columns

Column naming convention: `jointName_px`, `jointName_vx`, `jointName_ax`, `jointName_jx`, `jointName_py`, ..., `jointName_jz`.

9 Conclusion

This milestone successfully implements both the Linear Kalman Filter and Extended Kalman Filter in C++17 for full-body 3D human gait estimation. The implementations are built entirely from first principles using the matrices derived in Milestone 1, without any external linear-algebra libraries.

The four key design decisions — Cholesky-based solvers, heap memory allocation, Joseph-form covariance updates, and manual `arctan2` approximation — collectively ensure numerical stability, correct memory behaviour, and compliance with implementation requirements across all 3040 time steps and 23 joints.

The LKF and EKF both converge rapidly from the high-uncertainty initial state and produce smooth, physically plausible estimates of position, velocity, acceleration, and jerk for every joint. The EKF correctly handles the nonlinear spherical measurement model with angle-wrapping in the innovation step, producing results consistent with the LKF in Cartesian space.

9 References

1. R. E. Kalman, “A new approach to linear filtering and prediction problems,” *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.
2. A. Becker, *Kalman Filter from the Ground Up*, 2023.
3. X. R. Li and V. P. Jilkov, “Survey of maneuvering target tracking. Part I: Dynamic models,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 39, no. 4, pp. 1333–1364, Oct. 2003.
4. M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions*, Dover, 1965, Eq. 4.4.47.
5. NASA Technical Report, “A Discrete-Time Constant Jerk Tracking Model for Maneuvering Targets,” 2018.
6. S. Julier and J. Uhlmann, “Unscented filtering and nonlinear estimation,” *Proc. IEEE*, vol. 92, no. 3, pp. 401–422, Mar. 2004.
7. T. Lefebvre et al., “Kalman filters for non-linear systems: a comparison of performance,” *Int. J. Control*, vol. 77, no. 7, pp. 639–653, 2004.